

Name:

ID:

CS4575/CS6025: Parallel Computing

Winter 2026

Assignment 1: OpenMP Matrix Multiplication + SIMD + Performance Analysis

February 05, 2026

Weight: 8% of Final Grade

Due: 3 weeks from today (submit by **midnight**)

Work Type: Individual

Language: C (GCC + OpenMP)

Topic Coverage: Work decomposition → parallel loops → scoping → scheduling → reduction/atomic/critical → collapse → tasks/taskgroup → SIMD → speedup & efficiency

1) Learning Outcomes

By completing this assignment, you will be able to:

1. Implement a correct **serial baseline** and multiple **OpenMP versions** of the same program.
2. Apply OpenMP correctly:
 - parallel, for, single, task, taskgroup, barrier (if needed)
 - data scoping: shared, private, firstprivate, lastprivate, default(none)
 - scheduling: schedule(static), schedule(dynamic, chunk)
 - synchronization: reduction, atomic, critical
 - nested loops: collapse(2)
 - SIMD vectorization: simd, parallel for simd
3. Measure performance with sound methodology and compute:
 - **Speedup** and **efficiency** (thread efficiency and SIMD comparisons)
4. Explain and justify design choices based only on lecture concepts.

2) Problem Statement

You will write a complete C program that computes:

Core Kernel: Matrix Multiplication

Given two square matrices **A** and **B** (size **N×N**, type double), compute:

$$C = A \times B$$

Secondary Analytics

After computing **C**, your program must also compute:

1. sumC = sum of all elements in C
2. maxC = maximum value in C
3. checksum = a deterministic integer checksum based on C
 - **Important: The checksum in this assignment is NOT a real data-integrity checksum**

Name:

ID:

- Its purpose is purely educational.
- The checksum requirement exists only to create a situation where:
 - multiple threads must update the same shared variable, and
 - the update cannot be expressed cleanly using an OpenMP reduction.
- This allows you to practice and evaluate:
 - `#pragma omp atomic`
 - `#pragma omp critical`
 - and to compare their performance.
- **Definition Used in This Assignment**
 - You must compute a checksum value defined as:
 - `checksum += (long long)(C[i*N + j] * 1000.0) % 100000;`
 - for every element in the result matrix C.
 - This formula has no special mathematical meaning. It is intentionally chosen to:
 - be deterministic,
 - depend on every element of C,
 - require synchronized updates,
 - not be reducible using reduction.
- **What You Must Implement**
 - You are required to compute this checksum in **two separate ways**:
 - Using `#pragma omp atomic`
 - Using `#pragma omp critical`
 - and to compare their performance.

3) Input / Output Requirements

Command Line

Your executable must accept:

`./a1 N mode`

- N = matrix dimension (e.g., 512, 1024, 2048, ...)
- mode selects which implementation to run (see Section 5)

Example:

`./a1 2048 3`

Output

Your program must print:

- N, mode name

Name:

ID:

- number of threads used (from OpenMP runtime)
- timing for the kernel and total runtime
- sumC, maxC, and checksum
- speedup and efficiency (when applicable)

Your output must be clean and readable (avoid interleaved printing from multiple threads)

4) Data Generation and Memory Rules

Allocation

You must allocate matrices dynamically in **contiguous memory** (1D array), and use indexing.

Initialization

You must generate A and B internally (no file I/O). Use deterministic formulas so correctness can be verified:

$A[i*N + j] = \sin(i) * \cos(j) + \sqrt{i + j + 1};$

$B[i*N + j] = \cos(i) * \sin(j) + \sqrt{i + j + 2};$

(Use -lm when compiling.)

5) Required Implementations

You must implement **all** of the following modes in the same program.

Mode 0: Serial Baseline (No OpenMP at all)

- Standard triple-loop multiplication
- Then compute sumC, maxC, checksum serially

Mode 1: OpenMP Threads (Work-sharing loops + correct scoping)

- Use `#pragma omp parallel` for on the multiplication (outer loop or nested using collapse where appropriate)
- Correct scoping required: loop indices private; shared matrices shared; per-thread accumulators private/reduction, etc.
- **Must include at least one loop with:**
 - `schedule(static)` and another run with `schedule(dynamic, chunk)` (same code, just change schedule)

Mode 2: OpenMP Threads + collapse(2)

- Use:

`#pragma omp parallel for collapse(2)`

for a nested loop version (typically i and j loops).

Mode 3: Synchronization Comparison (Atomic vs Critical)

For computing the **checksum**, you must implement **two versions**:

- **Mode 3A:** uses `#pragma omp atomic`

Name:

ID:

- **Mode 3B:** uses `#pragma omp critical`

The checksum should be (can be something like):

```
checksum += (long long)(C[i*N + j] * 1000.0) % 100000;
```

This is intentionally not reducible cleanly in floating-point form. Use synchronization.

Mode 4: Task-Based Parallelism

You must implement matrix multiplication using **tasks**:

- Use:
 - `#pragma omp parallel`
 - `#pragma omp single`
 - Generate tasks for work units (e.g., blocks of rows, or tiles)
 - Use `taskgroup` to ensure all tasks complete before moving on

Example approach:

- Create tasks per **row block** (e.g., 16 rows per task) OR per **tile** (e.g., 32×32 tile).

6) SIMD Requirements

You must implement explicit OpenMP SIMD vectorization for **one loop that does real work**.

Required SIMD modes

You must implement and time:

Mode 5: SIMD Only (no threading)

- Force **1 thread**, but use:

```
#pragma omp simd
```

on a loop that is vectorizable.

Mode 6: Threads + SIMD

- Use:

```
#pragma omp parallel for simd
```

(or separate `parallel for` with an inner `simd` loop if appropriate)

Rules

- SIMD must be explicit using OpenMP directives (not intrinsics).
- You may still compile with `-O3`, but the SIMD requirement is satisfied only if you actually use `omp simd` / `parallel for simd`.

7) Performance Experiments

Timing tool

Use `omp_get_wtime()` for timing.

Runs

For performance reporting, you must test:

Name:

ID:

Matrix sizes

At least **3 sizes**:

- Small / Medium / Large (example: 512, 1024, 2048. choose based on your machine)

Thread counts

At least:

- 1, 2, 4, 8, 12, 16, 20 threads.

Repetitions

Run each configuration **10 times** and report the **average**, discarding the first run .

8) Required Metrics: Speedup and Efficiency

For each N and each thread count p:

Speedup

$$S(p) = T(1)/T(p)$$

Efficiency

$$E(p) = S(p)/p$$

You must compute these for:

- Threads-only (Mode 1 or 2)
- Tasks (Mode 4)
- Threads+SIMD (Mode 6)

SIMD comparison requirement

You must also compare:

- Mode 1 (threads only) vs Mode 6 (threads + SIMD)
- Mode 0 (serial) vs Mode 5 (SIMD-only)

9) Written Report (PDF, recommended 4–6 pages)

Your report must include:

A) Design Summary (practical focus)

- Which loops you parallelized and why
- Your scoping decisions (examples of private, firstprivate, etc.)
- What you used and why for sum/max
- Where/why you used atomic vs critical
- Why you chose your schedule and chunk size

B) Results Tables

Include at least:

1. Runtime tables (avg of 10 runs)
2. Speedup & efficiency tables

Name:

ID:

A required format example:

N	Mode	Threads	Schedule	Time(s)	Speedup	Efficiency
---	------	---------	----------	---------	---------	------------

C) Plots

1. Speedup vs threads
2. Efficiency vs threads

D) Discussion (theory-backed, but brief)

You must answer:

1. Static vs dynamic scheduling: which was better and why?
2. Why does efficiency drop as thread count increases?
3. Atomic vs critical: which was faster for checksum, and why?
4. Tasks vs loops: when did tasks help/hurt?
5. SIMD: which part benefited most, and did SIMD reduce the need for many threads?

10) Build and Run Requirements

Compilation

You do not need to provide a Makefile, but the build must work as:

```
gcc -O3 -fopenmp a1.c -lm -o a1
```

Platform

Must run on Linux.

11) Submission Package

Submit a ZIP:

A1_LastName_FirstName.zip

- └─ a1.c
- └─ Makefile (if applicable)
- └─ report.pdf
- └─ README.md

README must include:

- compile command
- example run commands
- machine specs you used for experiments (CPU model if possible)
- note of which N values you used

12) Academic Integrity

Name:

ID:

Follow the course policy. You must be able to explain your code and decisions. Keep an authorship trail (e.g., incremental versions or Git history; share the Git repository if you are using one) and include runtime evidence screenshots, as your course policy requires.